

Probeklausur Systemprogrammierung 2024/2025

Beachte unterschied zwischen Pseudocode und C Verwenden sie für die Beantwortung möglichst den eingeplanten Platz

Aufgabe 1.) Datentypen (1+2+1 Punkte)

1.1 Weshalb wird empfohlen `stdint.h` bzw `inttypes.h` zu inkludieren und diese Typen zu verwenden?

1.2 Was ist Padding? Unter welchen Umständen hat Padding in C einen Einfluss auf die Programmierung?

1.3 Was ist Alignment und wie hängt es mit Padding zusammen?

Aufgabe 2.) Lieblingsfarbe (3 Punkte)

Definieren sie eine RGB-Farbe (Komponenten sind 8 bittig) über eine struct.

Definieren sie auch eine union, auf die Farbe als 32-Bit Wert (raw) zugreifen.

Schreiben sie eine Funktion in C, welche sowohl den RGB- als auch den raw-Wert einer als Parameter übergebenen Farbe per `printf()` in der Konsole ausgibt.

Aufgabe 3.) Bitmaskierung (2 + 4 Punkte)

3.1 Erklären sie die Unterschiede bei der Verwendung von Bitmaskierung und Bitfields. Geben Sie ein Beispiel analog in beiden Varianten an

3.2 Bei der seriellen Kommunikation (RS-232) ist ein gerades Paritätsbit gesetzt, wenn
die Anzahl gesetzter Bits in einem Byte inklusive Paritätsbit gerade,
also ohne dass Paritätsbit
ungerade ist. Schreiben Sie eine Funktion in C, welche als Parameter ein Byte (8 Bit unsigned) erhält
und das berechnete Paritätsbit für gerade Parität zurückgibt.

Aufgabe 4.) Dynamische Speicherverwaltung (5 Punkte)

in der dynamischen Speicherverwaltung der Übungsblätter 2/3/4 fehlt eine wesentliche

Funktion: `mem_get_Sizes(...)`. Diese Funktion soll die Größe des allokierten und die

Größe des freien Speichers zurückgeben. Dazu müssen alle Speicherblöcke durchwandert und deren

Größe entsprechend gezählt werden. Der Speicher, den die Header verbrauchen wird jedenfalls zum allokierten Speicher gezählt.

Die Checksumme kann beim durchlaufen ignoriert werden.

```
struct Header{ uint8_t flags; uint8_t checksum; uint16_t length;}  
Heap ist im globalen Array wie folgt:  
{Heapsize 1024; Uint8_t gHeap[Heapsize]}
```

Aufgabe 5.) Multitasking (2 + 2 + 4 + 2 + 2 Punkte)

5.1 Semaphor

Was ist ein Semaphor? Wie funktioniert er? Was bedeuten die Aufrufe P() und V()

5.2 Semaphor in FreeRtos

Erklären sie folgende FreeRtos Semaphor- Funktionen: Was bedeuten die Parameter, was die Rückgabetypen

```
SemaphorHandle_t xSemaphoreCreateCounting(UBaseType_t uxMaxCount,  
UBaseType_t uxInitialcount);
```

```
xSemaphoreGive(SemaphoreHandle_t xSemaphore);
```

```
XSemaphoreTake(SemaphoreHandle_t xSemaphore, TickType_t xTicksToWait);
```

5.3 Producer/Consumer

Gegeben sind zwei Taskfunktionen, eine für eine Producer und eine für eine Consumer. Der Producer soll le 30ms ein Ereignis

"Produzieren" und an den Consumer senden. Der Consumer soll alle 30ms per printf() auf die Konsole ausgeben

```
void ProducerTaskFunc(void * pvParameters){}
```

```
void consumerTaskFunc(void + pvParameters){}
```

5.4 Scheduling

Skizzieren Sie das Scheduling in einem Diagramm, Nehmen sie an, dass eine Zeitscheibe (System Tick) 10ms beträgt ein

Idle-Task mit niedriger Priorität Prozessorzeit konsumiert, wenn alle anderen Tasks blockiert sind.

5.5 Producer/Consumer mit Produkten

Angenommen, der Producer möchte Produkte vom Typ "struct Product" an den Consumer senden. Welche Datenstruktur ist dann geeignet?

Skizzieren sie eine entsprechende Änderung des Producer/Consumer-Codes (Pseudocode ist hier erlaubt).